NAPAC 2025 · SACRAMENTO, CA · JACOW PP. 65–67 · POSTER MOP010

`AtomicAndPhysicalConstants.jl`: A Fast Julia Package to Access Particle Properties and Fundamental Constants

L. Li, **D. Sagan** · CLASSE, Cornell University, Ithaca, NY, USA
D. Abell · University of Maryland, College Park, College Park, MD, USA
A. Coxe · Virginia Commonwealth University, Richmond, VA, USA

TL;DR A Julia package for the SciBmad project that provides physical constants, subatomic particle properties, and atomic isotope data from CODATA, NIST, and PDG. One `@APCdef` macro call fixes units and numeric types up front, so exported functions are type-stable and core calls run in under 1 microsecond.

Work supported by Brookhaven Science Associates, LLC under Contract Nos. DE-SC0012704 and DE-SC0018008 with the U.S. Department of Energy.

MOTIVATION

Beam simulation needs constants that are fast, stable, and traceable

1 CONTEXT

- **Bmad**, developed at Cornell's Laboratory for Elementary Particle Physics, has long been a key framework for simulating charged-particle dynamics.
- **SciBmad** is a new Julia-based simulation environment that uses Julia's performance, parallelism support, and native multiple dispatch.
- Within SciBmad, this package fills the need for **fast and stable access to physical constants**.

2 THE PERFORMANCE TRAP

- Simulations want different **unit systems** and **numeric types**, but custom units and types usually lead to type instability and slow run time.
- The typical approach is a getter that **converts units and type at run time**. This adds computation and makes the function **type-unstable**.
- If the return type is not decided at compile time, the compiler cannot optimize the program, which costs more memory and time.

3 EXISTING PACKAGES

- **PhysicalConstants.jl**, one of the most widely used Julia packages, provides reliable static constants with uncertainty support.
- AtomicAndPhysicalConstants.jl adds a **direct interface** for constants and an option to **customize units and variable types**.
- Its `Species` type wraps particle attributes such as mass and charge for simpler access.

doi.org/10.18429/JACoW-NAPAC2025-MOP010

Aggregated data, configured once by a macro

1 DATA SOURCES

CODATA CONSTANTS

Loads the most recent release by default (currently CODATA 2022); earlier releases such as CODATA 2018 stay available through versioned submodules.

NIST ISOTOPES

NIST keeps no history of its isotope datasets, so isotope data reflect the most recent release.

PDG PIONS

π^0 and $\pi^{\pm}$ properties are downloaded directly from the PDG API.

2 `@APCdef` KEYWORDS

KEYWORD	CHOICES
`unitsystem`	`ACCELERATOR`, `MKS` (SI), `CGS`, or a custom five-tuple of Unitful units
`unittype`	`Float64`, `Unitful.Quantity`, or `DynamicQuantity.Quantity`
`name`	Name of the named tuple holding the constants (default `APC`)
`tupleflag`	`false` defines constants as global variables in the current namespace

Invoked once during package initialization; a second invocation raises an error. It sets units and types for all constants and for `Species` mass and charge.

3 GENERATE, DON'T CONVERT

CONFIGURE
Choose units and numeric type in a single macro call

↓

GENERATE
Constants and getters such as `massof()`, `chargeof()` are written into the namespace with a fixed unit and type

↓

RUN
No unit conversion; return types are known at compile time

Trade-off: the macro call itself is slower, but the program is fast at run time.

doi.org/10.18429/JACoW-NAPAC2025-MOP010

One type for particles and atomic isotopes

1 CONSTRUCTING A `Species`

- `Species(::String)` takes an **OpenPMD**-standard particle name. Fields: `name`, `mass`, `charge`, `spin`, `moment`, `iso`.
- **Subatomic particles:** exact, case-sensitive names; antiparticles use the “anti-” prefix. 14 subatomic particles are supported.
- **Atoms:** an atomic symbol, an optional isotope mass number (prefix `13C` or superscript ^{13}C), and an optional charge (`+`, `++`, `+n`, or negatives).
- No isotope given: mass is the abundance-weighted mean (per NIST). No charge given: neutral.
- **Null species:** a placeholder for a field that must exist but is not yet assigned.

`ATOM` `HADRON` `LEPTON` `PHOTON` `NULL`

2 PROPERTIES AND DERIVED QUANTITIES

Getters return mass m, charge q, atomic number Z, and spin S in the units and types chosen by `@APCdef`. Derived quantities:

$$g = \frac{2m\mu}{Sq}$$ **Gyromagnetic ratio**; μ is the magnetic moment

$$g_a = \frac{g-2}{2}$$ **Gyromagnetic anomaly**, for leptonic species

$$g_n = \frac{gZm_p}{m}$$ **Nucleon-normalized g-factor**, for baryonic species; m_p is the proton mass

doi.org/10.18429/JACoW-NAPAC2025-MOP010

USAGE

From configuration to particle data in a few lines

1 CONSTANTS

SESSION A · DEFAULTS (ACCELERATOR UNITS, FLOAT64)

```
julia> using AtomicAndPhysicalConstants
julia> @APCdef
julia> APC.C_LIGHT        # m/s
2.99792458e8
julia> APC.E_CHARGE       # elementary charge
1.0
julia> e = Species("electron")
julia> massof(e)          # eV/c²
510998.95069
```

SESSION B · UNITFUL, MKS, CONSTANTS AS GLOBALS

```
julia> @APCdef unittype = Unitful unitsystem = MKS tupleflag = false
julia> E_CHARGE
1.602176634e-19 C
```

2 SPECIES STRINGS

```
Species("anti-proton")
Species("13C+")     # C-13, +1
Species("¹⁵N³⁻")    # N-15, -3
Species("Al+3")     # mean Al, +3
Species("Null")     # placeholder
```

Two separate sessions: `@APCdef` is called once per session, so each configuration starts fresh.

Examples and outputs are taken from the package documentation for the `@APCdef` releases (v0.7.2 README; units and species pages, v0.7.2–v0.8.2). Comments are added.

doi.org/10.18429/JACoW-NAPAC2025-MOP010

QUALITY CONTROL AND PERFORMANCE

Tested in CI, benchmarked for type stability

1 TESTING

- An extensive, modular test suite validates `@APCdef` configurations and `Species` constructor parsing rules.
- All tests run on **GitHub Actions** as a continuous integration (CI) workflow.

2 BENCHMARKING

Goal: ensure **type stability** (a variable's type is decided at compile time) and improve speed.

- **BenchmarkTools.jl** measured memory allocations and execution time of `@APCdef`, species construction, and getters such as `massof()` and `chargeof()`.
- **JET.jl** detected and eliminated dynamic dispatches and type instabilities.

3 RESULTS

$< 1\ \mu s$

Core function run time, including `@APCdef` and `Species()`

> "All exported functions are verified as type-stable using JET.jl, and benchmarks confirm minimal allocations during constant access."
>
> Quality Control, p. 67

doi.org/10.18429/JACoW-NAPAC2025-MOP010

SUMMARY

A type-stable, unit-aware foundation for SciBmad

1 TAKEAWAYS

- A **type-stable, unit-aware** interface to physical constants.
- Authoritative datasets with pinnable historical versions give **numerical traceability**.
- One macro configures unit systems and variable types, integrating **Unitful.jl** and **DynamicQuantities.jl**.
- `Species` simplifies work with subatomic particles and isotopes.

2 AT A GLANCE

16 + 14

Common constants and subatomic particles, plus all NIST isotope data

3 × 3

Unit systems (MKS, CGS, ACCELERATOR) × numeric types (Float64, Unitful, DynamicQuantities)

< 1 μs

Core function run time, including `@APCdef` and `Species()`

3 FUTURE WORK

- **Expanded data coverage:** more physics constants.
- **Performance tuning:** faster `@APCdef` and `Species` initialization.
- **Community-driven evolution:** updates from GitHub issues, plus new CODATA, NIST, and PDG releases.

CODE

`github.com/bmad-sim/AtomicAndPhysicalConstants.jl`

doi.org/10.18429/JACoW-NAPAC2025-MOP010